

eda_programm_summury_v1.2

December 12, 2025

1 EDA - programm_summury.csv

1.1 Chargement et inspection initiale

Objectif Charger les fichiers du dataset **600K+ Fitness Exercise & Workout Program Dataset** et obtenir une première vue d'ensemble :

- dimensions de chaque fichier (lignes / colonnes) ;
- aperçu des premières lignes (cohérence des champs) ;
- typage des variables et complétude (valeurs manquantes) ;
- présence éventuelle de doublons ;
- liste complète des colonnes disponibles.

Cette étape valide que le corpus est exploitable avant nettoyage et préparation NLP pour le projet **TrAIn.me**.

Interprétation attendue À l'issue de cette première inspection, on commentera :

- la taille du dataset (volume “exercice” vs volume “programmes”) ;
- la nature des variables : textes (title/description), catégories (level/goal/equipment), structure (week/day), paramètres d'entraînement (sets/ reps/intensity), métadonnées (timestamps) ;
- la présence de valeurs manquantes et leur ordre de grandeur ;
- l'existence (ou non) de doublons stricts ;
- la cohérence générale des premières lignes (valeurs lisibles, structure logique, champs textuels non vides).

Ce diagnostic sert de référence avant les étapes de nettoyage, normalisation et structuration pour le Deep Learning / NLP.

1.1.1 Importation des librairies essentielles

```
[1]: import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import LogNorm
import textwrap
from matplotlib.ticker import PercentFormatter
import seaborn as sns
```

```

import kagglehub
import prince

# === Localiser automatiquement le package 'themes' en remontant l'arborescence
↳===
from pathlib import Path
import sys

root = Path.cwd()          # ex: .../DuckDB/eda/life_style_data
themes_root = None

for p in [root, *root.parents]:  # remonte: life_style_data -> eda -> DuckDB
    ↳-> ...
    if (p / "themes").is_dir():
        themes_root = p
        break

if themes_root is None:
    raise FileNotFoundError(
        f"Impossible de trouver le dossier 'themes' en partant de {root}. "
        "Vérifie l'arborescence (il doit exister un dossier 'themes/' quelque
        ↳part au-dessus)."
    )

if str(themes_root) not in sys.path:
    sys.path.insert(0, str(themes_root))

print("Ajouté au PYTHONPATH :", themes_root)
print("Contenu de", themes_root, ":", [x.name for x in (themes_root).iterdir()])

# Thème personnalisé du projet (uniquement pour les notebooks)
from themes.theme_train_me import set_trainme_theme

```

```

c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\conda\Lib\site-
packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
    from .autonotebook import tqdm as notebook_tqdm

```

```

Ajouté au PYTHONPATH :
c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\src\notebooks
Contenu de c:\Users\fbac\Desktop\Projets\Dev\GitHub\train.me\src\notebooks :
['dataset_fusion', 'eda', 'model_training', 'preprocessing', 'themes',
'__init__.py', '__pycache__']

```

1.1.2 Configuration d’affichage pour plus de lisibilité

```
[2]: pd.set_option('display.max_columns', None)
pd.set_option('display.width', 120)
```

1.1.3 Thème “TrAIn.me”

Il applique un fond dark propre, une grille discrète, des ticks lisibles, et une palette bleus/cians cohérente.

```
[3]: # =====
# Thème TrAIn.me (Seaborn/Matplotlib)
# =====
# Booléen de contrôle : True = thème TrAIn.me (dark), False = style clair
# ↪ "ticks"
USE_DARK_THEME = True # modifie ici selon le contexte

# Activer le thème au début du notebook :
palette_trainme = set_trainme_theme(context="talk", font_scale=1.05,
# ↪ use_dark=True)
palette_trainme
```

```
[3]: ['#00E5FF', '#00BFFF', '#0A84FF', '#1E90FF', '#00FFFF', '#64D8FF']
```

1.1.4 Chargement du dataset

```
[6]: # Téléchargement du dataset depuis Kaggle
# Identifiant Kaggle attendu : "adnanelouardi/
# ↪ 600k-fitness-exercise-and-workout-program-dataset"
dataset_path = kagglehub.dataset_download("adnanelouardi/
# ↪ 600k-fitness-exercise-and-workout-program-dataset")
print("Dataset téléchargé dans :", dataset_path)

# Recherche des CSV disponibles
csv_files = sorted([f for f in os.listdir(dataset_path) if f.endswith(".csv")])
print("Fichiers CSV trouvés :", csv_files)

# On cible explicitement les deux fichiers décrits dans la page Kaggle
path_programs = os.path.join(dataset_path, "program_summary.csv")

# Chargement
df = pd.read_csv(path_programs)

print("Chargement terminé.")
print(f"program_summary.csv : {df.shape[0]:,} lignes | {df.shape[1]:,}
# ↪ colonnes")
```

Dataset téléchargé dans :
C:\Users\fback\.cache\kagglehub\datasets\adnanelouardi\600k-fitness-exercise-and-workout-program-dataset\versions\1
Fichiers CSV trouvés : ['program_summary.csv',
'programs_detailed_boostcamp_kaggle.csv']
Chargement terminé.
program_summary.csv : 2,598 lignes | 10 colonnes

1.1.5 Aperçu général des premières lignes

```
[7]: display(df.head())
```

	description \	title		
0	(MASS MONSTER) High Intensity 4 Day Upper Lowe... Build tones of muscular with this high intensi...			
1	(NOT MY PROGRAM)SHJ Jotaro			
2	1 PowerLift Per Day Powerbuilding 5 Day Bro Split Based off of Andy Baker's KCS (Kingwood Streng...			
3	10 Week Mass Building Program This workout is designed to increase your musc...			
4	10 week deadlift focus			
	Increase deadlift			
	goal equipment \	level		
0	['Intermediate'] ['Muscle & Sculpting', 'Bodyweight Fitness'] Full Gym			
1	['Advanced', 'Intermediate'] ['Bodybuilding'] Full Gym			
2	['Beginner', 'Novice', 'Intermediate'] ['Athletics', 'Powerlifting', 'Powerbuilding'] Full Gym			
3	['Intermediate', 'Advanced'] ['Powerbuilding'] Garage Gym			
4	['Intermediate', 'Advanced'] ['Powerbuilding', 'Powerlifting', 'Bodybuildin...'] Full Gym			
	program_length	time_per_workout	total_exercises	created
	last_edit			
0	12.0	90.0	384	2024-01-20 10:23:00
	2025-06-29 12:39:00			
1	8.0	60.0	224	2024-07-08 02:28:00
	2025-06-18 09:15:00			
2	6.0	90.0	237	2025-04-23 09:21:00
	2025-06-18 11:55:00			

```

3          10.0          70.0          280  2024-09-07 03:44:00
↳2025-06-18 08:01:00
4          10.0          80.0          356  2024-12-23 03:13:00
↳2025-06-18 12:19:00

```

1.1.6 Informations générales sur le typage et la complétude

```
[8]: df_info = df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2598 entries, 0 to 2597
Data columns (total 10 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   title                 2598 non-null   object
 1   description           2594 non-null   object
 2   level                 2598 non-null   object
 3   goal                  2598 non-null   object
 4   equipment             2597 non-null   object
 5   program_length        2597 non-null   float64
 6   time_per_workout      2598 non-null   float64
 7   total_exercises       2598 non-null   int64
 8   created               2597 non-null   object
 9   last_edit             2596 non-null   object
dtypes: float64(2), int64(1), object(7)
memory usage: 203.1+ KB

```

1.1.7 Statistiques descriptives des variables numériques

```
[9]: display(df.describe().T)
```

```

              count      mean      std   min   25%   50%   75%
↳max
program_length  2597.0    8.812476   4.185403   1.0    5.0    8.0   12.00
↳18.0
time_per_workout  2598.0   69.035412  24.394798  10.0   60.0   60.0   90.00
↳180.0
total_exercises  2598.0  232.884142  208.123873   1.0  108.0  192.5  307.75
↳5040.0

```

1.1.8 Analyse des valeurs manquantes et doublons

```

[10]: missing_values = df.isnull().sum().sort_values(ascending=False)
      duplicates = df.duplicated().sum()

print("\n--- Valeurs manquantes par colonne (top 10) ---")
display(missing_values.head(10))

```

```
print(f"\nNombre total de doublons détectés : {duplicates}")
```

--- Valeurs manquantes par colonne (top 10) ---

```
description      4
last_edit        2
equipment         1
program_length   1
created          1
title            0
level            0
goal             0
time_per_workout 0
total_exercises  0
dtype: int64
```

Nombre total de doublons détectés : 0

1.1.9 Liste complète des colonnes disponibles

```
[11]: print("\n--- Liste des colonnes du dataset ---")
      print(df.columns.tolist())
```

--- Liste des colonnes du dataset ---

```
['title', 'description', 'level', 'goal', 'equipment', 'program_length',
'time_per_workout', 'total_exercises', 'created', 'last_edit']
```

1.2 Statistiques descriptives et distribution des variables

Objectif Obtenir une vue d’ensemble chiffrée des variables numériques du dataset **program_summary.csv** :

- résumer la distribution des variables disponibles (durée du programme, durée par séance, volume total d’exercices) ;
- repérer les ordres de grandeur, la dispersion et les éventuelles valeurs extrêmes ;
- préparer l’identification d’outliers et la sélection de variables pertinentes pour la suite de l’analyse.

Interprétation attendue À l’issue de cette section, l’interprétation devra mettre en évidence :

- les valeurs centrales des variables clés (durée typique des programmes, temps moyen par séance, volume global d’exercices) ;
- la dispersion (écarts-types, min/max, quartiles) afin d’identifier des programmes “courts” vs “longs”, et des programmes “légers” vs “très volumineux” ;
- la présence éventuelle de valeurs extrêmes ou atypiques (ex. volume d’exercices très élevé) pouvant nécessiter un traitement spécifique ;

- les premières tendances structurantes utiles pour le projet **TrAIn.me** (plages réalistes à cibler lors de la génération de programmes).

1.2.1 Activation du thème TrAIn.me

```
[12]: if USE_DARK_THEME:
    palette_trainme = set_trainme_theme(context="talk", font_scale=1.05,
    ↪use_dark=True)
else:
    sns.set_style("ticks")           # style clair sobre, axes renforcés
    sns.set_context("talk", font_scale=1.05)
    palette_trainme = sns.color_palette("deep") # palette par défaut sobre
    print("Style activé : 'ticks' → sobre, axes renforcés (présentation pro)")

# Exemple de vérification :
print("Palette active :", palette_trainme.as_hex() if hasattr(palette_trainme,
    ↪"as_hex") else palette_trainme)
```

Palette active : ['#00E5FF', '#00BFFF', '#0A84FF', '#1E90FF', '#00FFFF', '#64D8FF']

1.3 Statistiques descriptives globales

```
[13]: # --- Colonnes numériques disponibles ---
num_cols = df.select_dtypes(include="number").columns.tolist()
print(f"Variables numériques détectées ({len(num_cols)}) : {num_cols}")

# Statistiques descriptives de base
if num_cols:
    stats_num = df[num_cols].describe().T
    display(stats_num)
else:
    print("Aucune variable numérique détectée.")

# Quelques repères métier (si les colonnes existent)
if "program_length" in df.columns:
    print(f"\nDurée moyenne du programme (semaines) : {df['program_length'].
    ↪mean():.2f}")

if "time_per_workout" in df.columns:
    print(f"Temps moyen par séance (minutes) : {df['time_per_workout'].mean():.
    ↪2f}")

if "total_exercises" in df.columns:
    print(f"Nombre total moyen d'exercices par programme :
    ↪{df['total_exercises'].mean():.2f}")
```

Variables numériques détectées (3) : ['program_length', 'time_per_workout',

	count	mean	std	min	25%	50%	75%	
↪max								
program_length	2597.0	8.812476	4.185403	1.0	5.0	8.0	12.00	↪
↪18.0								
time_per_workout	2598.0	69.035412	24.394798	10.0	60.0	60.0	90.00	↪
↪180.0								
total_exercises	2598.0	232.884142	208.123873	1.0	108.0	192.5	307.75	↪
↪5040.0								

Durée moyenne du programme (semaines) : 8.81

Temps moyen par séance (minutes) : 69.04

Nombre total moyen d'exercices par programme : 232.88

1.3.1 Distribution des variables clés

```
[16]: # Variables numériques pertinentes (ici : toutes sont métier)
num_vars = list(num_cols)
print(f"Variables numériques représentées : {num_vars}")

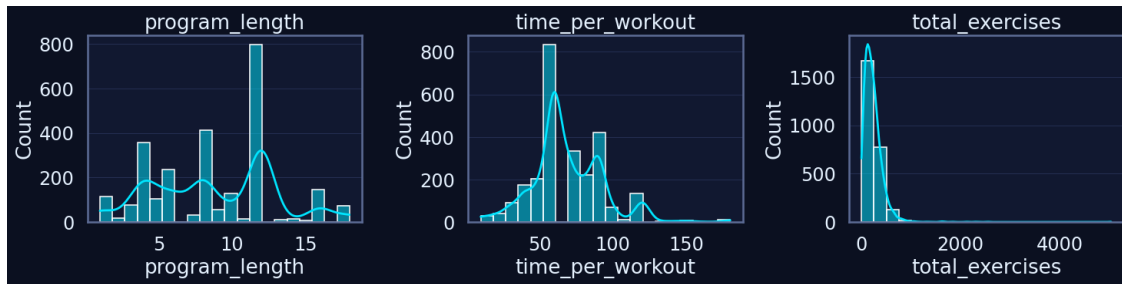
n = len(num_vars)
if n == 0:
    print("Aucune variable numérique pertinente à visualiser.")
else:
    n_cols = 3
    n_rows = int(np.ceil(n / n_cols))

    plt.figure(figsize=(5 * n_cols, 4 * n_rows))

    for i, col in enumerate(num_vars, 1):
        plt.subplot(n_rows, n_cols, i)
        sns.histplot(df[col].dropna(), bins=20, kde=True)
        plt.title(col)
        plt.grid(True, axis="y")

    plt.tight_layout()
    plt.show()
```

Variables numériques représentées : ['program_length', 'time_per_workout', 'total_exercises']

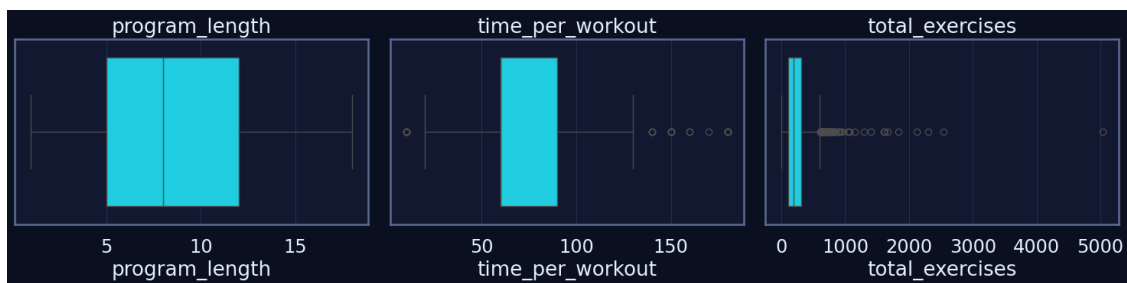


1.3.2 Boxplots pour repérer les outliers

```
[17]: if num_vars:
    plt.figure(figsize=(5 * len(num_vars), 4))

    for i, col in enumerate(num_vars, 1):
        plt.subplot(1, len(num_vars), i)
        sns.boxplot(x=df[col])
        plt.title(col)
        plt.grid(True, axis="x")

    plt.tight_layout()
    plt.show()
else:
    print("Aucun boxplot pertinent à afficher (pas de variable numérique_
    ↪métier).")
```



1.3.3 Statistiques descriptives enrichies

```
[18]: if num_cols:
    desc = df[num_cols].describe().T
    desc["missing_%"] = df[num_cols].isna().mean() * 100
    desc["zeros_%"] = (df[num_cols] == 0).mean() * 100
    desc["skew"] = df[num_cols].skew(numeric_only=True)
    desc["kurt"] = df[num_cols].kurt(numeric_only=True)
```

```

display(desc.sort_values("std", ascending=False))
else:
    print("Aucune variable numérique disponible pour des statistiques enrichies.
↵")

```

	count	mean	std	min	25%	50%	75%	
↵max missing_%	zeros_%	skew \						
total_exercises	2598.0	232.884142	208.123873	1.0	108.0	192.5	307.75	↵
↵5040.0	0.000000	0.0	7.099139					
time_per_workout	2598.0	69.035412	24.394798	10.0	60.0	60.0	90.00	↵
↵180.0	0.000000	0.0	0.727516					
program_length	2597.0	8.812476	4.185403	1.0	5.0	8.0	12.00	↵
↵18.0	0.038491	0.0	0.084756					

	kurt
total_exercises	125.265230
time_per_workout	2.039627
program_length	-0.733091

1.3.4 Détection rapide d'outliers (IQR)

```

[19]: iqr_report = {}

for col in num_cols:
    s = df[col].dropna()
    if s.empty:
        iqr_report[col] = 0
        continue

    Q1 = s.quantile(0.25)
    Q3 = s.quantile(0.75)
    IQR = Q3 - Q1

    outliers = df[(df[col] < Q1 - 1.5 * IQR) | (df[col] > Q3 + 1.5 * IQR)]
    iqr_report[col] = len(outliers)

iqr_df = pd.DataFrame.from_dict(iqr_report, orient="index",
↵columns=["outliers"])
display(iqr_df.sort_values("outliers", ascending=False))

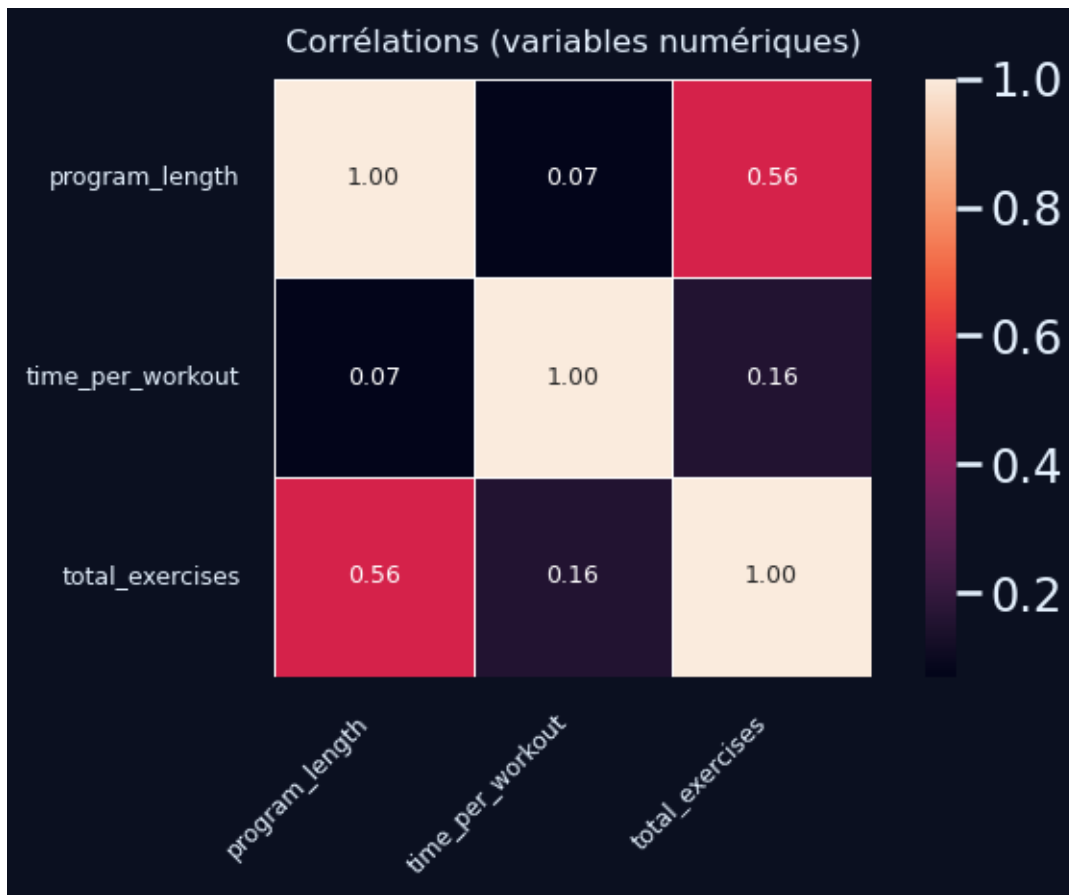
```

	outliers
total_exercises	85
time_per_workout	58
program_length	0

1.3.5 Aperçu corrélations (sous-ensemble lisible)

```
[20]: if len(num_cols) >= 2:
    subset_cols = num_cols # peu de colonnes → heatmap lisible
    corr = df[subset_cols].corr(numeric_only=True)

    plt.figure(figsize=(7, 5))
    sns.heatmap(
        corr,
        annot=True,
        fmt=".2f",
        square=True,
        cbar=True,
        linewidths=0.5,
        annot_kws={"size": 9},
    )
    plt.title("Corrélations (variables numériques)", fontsize=12, pad=10)
    plt.xticks(fontsize=9, rotation=45, ha="right")
    plt.yticks(fontsize=9, rotation=0)
    plt.tight_layout()
    plt.show()
else:
    print("Corrélation non pertinente : une seule variable numérique disponible.
    ↪")
```



1.4 Visualisation univariée

Objectif Compléter les statistiques descriptives par une analyse visuelle des distributions des variables numériques du dataset **program_summary.csv**.

L'objectif est de : - visualiser la forme des distributions (centrage, dispersion, asymétrie) ; - repérer visuellement d'éventuelles valeurs extrêmes (outliers) ; - confirmer les plages réalistes des variables structurantes (durée, temps par séance, volume d'exercices), afin de cadrer les futures étapes de préparation et de génération de programmes pour **TrAIIn.me**.

Interprétation attendue À l'issue de cette section, l'analyse devra : - décrire la distribution des variables numériques (formes dominantes, valeurs typiques) ; - identifier les variables potentiellement asymétriques (ex. "total_exercises" souvent très étalée) ; - signaler les points extrêmes susceptibles de perturber l'entraînement (programmes anormalement longs / volumineux) ; - proposer des pistes de traitement pour la suite (cap, clipping, filtrage, ou pondération) si nécessaire.

1.4.1 Sélection robuste des variables numériques à visualiser

```
[21]: # Colonnes numériques
num_cols = df.select_dtypes(include="number").columns.tolist()

# Filtre de robustesse : au moins 90% de valeurs non-nulles
min_non_null_ratio = 0.90
robust_num_cols = [
    c for c in num_cols
    if df[c].notna().mean() >= min_non_null_ratio
]

# Exclusion éventuelle d'identifiants techniques (si présents dans d'autres
↳ datasets)
excluded = {"id", "program_id", "parent_id"}
robust_num_cols = [c for c in robust_num_cols if c not in excluded]

print(f"Variables numériques détectées : {num_cols}")
print(f"Variables numériques retenues (robustes) : {robust_num_cols}")
```

Variables numériques détectées : ['program_length', 'time_per_workout', 'total_exercises']

Variables numériques retenues (robustes) : ['program_length', 'time_per_workout', 'total_exercises']

1.4.2 Visualisation des distributions univariées (hist + KDE)

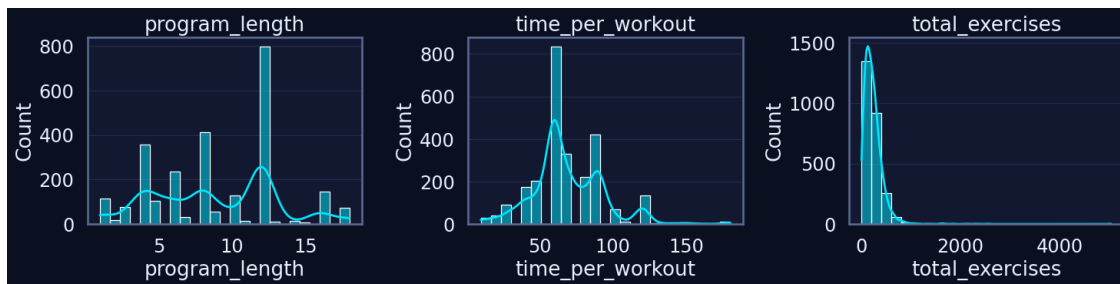
```
[22]: import matplotlib.pyplot as plt
import seaborn as sns

if not robust_num_cols:
    print("Aucune variable numérique robuste à visualiser.")
else:
    n = len(robust_num_cols)
    n_cols = 3
    n_rows = int(np.ceil(n / n_cols))

    plt.figure(figsize=(5 * n_cols, 4 * n_rows))

    for i, col in enumerate(robust_num_cols, 1):
        plt.subplot(n_rows, n_cols, i)
        sns.histplot(df[col].dropna(), bins=25, kde=True)
        plt.title(col)
        plt.grid(True, axis="y")

    plt.tight_layout()
    plt.show()
```



1.4.3 Boxplots univariés pour détection visuelle d'outliers

```
[23]: if not robust_num_cols:
    print("Aucun boxplot à afficher.")
else:
    plt.figure(figsize=(5 * len(robust_num_cols), 4))

    for i, col in enumerate(robust_num_cols, 1):
        plt.subplot(1, len(robust_num_cols), i)
        sns.boxplot(x=df[col].dropna())
        plt.title(col)
        plt.grid(True, axis="x")

    plt.tight_layout()
    plt.show()
```

